VLSI DESIGN TASK 1

Digital Arithmetic Logic Circuit Library Using Verilog HDL

Prepared for	Industry-Oriented Internship Program on VLSI Design and Semiconductor Engineering
Student	Vishnu Vamsi
Repository	github.com/vishnuvamsi396-gif/VLSI-Design-Task-1
Report date	2026-11-01

1. Objective

Implement a small reusable combinational RTL library and demonstrate the Task 1 workflow: digital logic design, synthesizable Verilog, self-checking testbenches, Icarus Verilog simulation, waveform generation, and organized version-controlled documentation.

2. Scope

The project includes basic logic gates, 2:1 and 4:1 multiplexers, half and full adders, a 4-bit ripple-carry adder, a parameterized unsigned adder, and a 4-bit arithmetic/logic mini-project unit. The assignment also introduces Linux-based development and the VLSI design flow; this introductory submission does not claim physical design, synthesis signoff, or chip fabrication.

ASIC and FPGA comparison

Criterion	ASIC	FPGA
Behavior	Fixed after fabrication	Reconfigurable after deployment
Up-front cost	Higher design and fabrication cost	Lower initial cost; no custom mask set
Typical use	Production SoCs and custom accelerators	Prototyping, instrumentation, adaptable networking
Main trade-off	Manufacturing commitment and longer iteration	Programmable resources can cost power, area, or speed

3. Design inventory

Design	Interface / purpose
Basic gates	AND, OR, XOR, NOT, NAND, NOR, XNOR; one-bit truth-table functions.
Multiplexers	2:1 one-bit data selection; 4:1 one-bit selection from a 4-bit input bus.
Half adder	Two one-bit inputs; sum and carry outputs.
Full adder	Two one-bit inputs plus carry-in; sum and carry-out.
4-bit ripple adder	Four cascaded full-adder stages with carry-in and carry-out.
Parameterized adder	Unsigned WIDTH-bit addition with explicit carry-out.
Mini-project unit	4-bit AND, OR, XOR, or addition selected by a 2-bit operation code.

4. Logic reference

The following truth table is the reference for the single-bit gate implementation. A labeled symbol diagram for the basic gates is included as `docs/diagrams/basic_logic_gates.svg` in the repository.

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

Boolean expressions

Half adder: SUM = A XOR B; CARRY = A AND B.
Full adder: SUM = A XOR B XOR Cin; Cout = AB + ACin + BCin.
2:1 mux: Y = (~S AND D0) OR (S AND D1).

Karnaugh-map simplifications

Function	Minterms	Minimal SOP
AND	m3	A.B
OR	m1, m2, m3	A + B
XOR	m1, m2	(~A.B) + (A.~B)

Mini-project operation map

operation	Result	Carry-out
00	a AND b	0
01	a OR b	0
10	a XOR b	0
11	unsigned a + b	Addition carry

5. VLSI context

A typical digital IC flow proceeds from specification and architecture through RTL, functional verification, synthesis, implementation (floorplanning, placement, clock-tree synthesis, routing), signoff, and fabrication. ASICs are custom-fabricated for a target product; FPGAs provide reconfigurable programmable logic and are commonly used for prototyping and adaptable products. This repository focuses on introductory RTL and functional simulation.

RTL and simulation workflow

Specification -> RTL modules -> self-checking testbenches -> iverilog compile -> vvp simulation -> pass/fail log and VCD -> GTKWave inspection -> screenshot evidence.

6. Verification plan and observed results

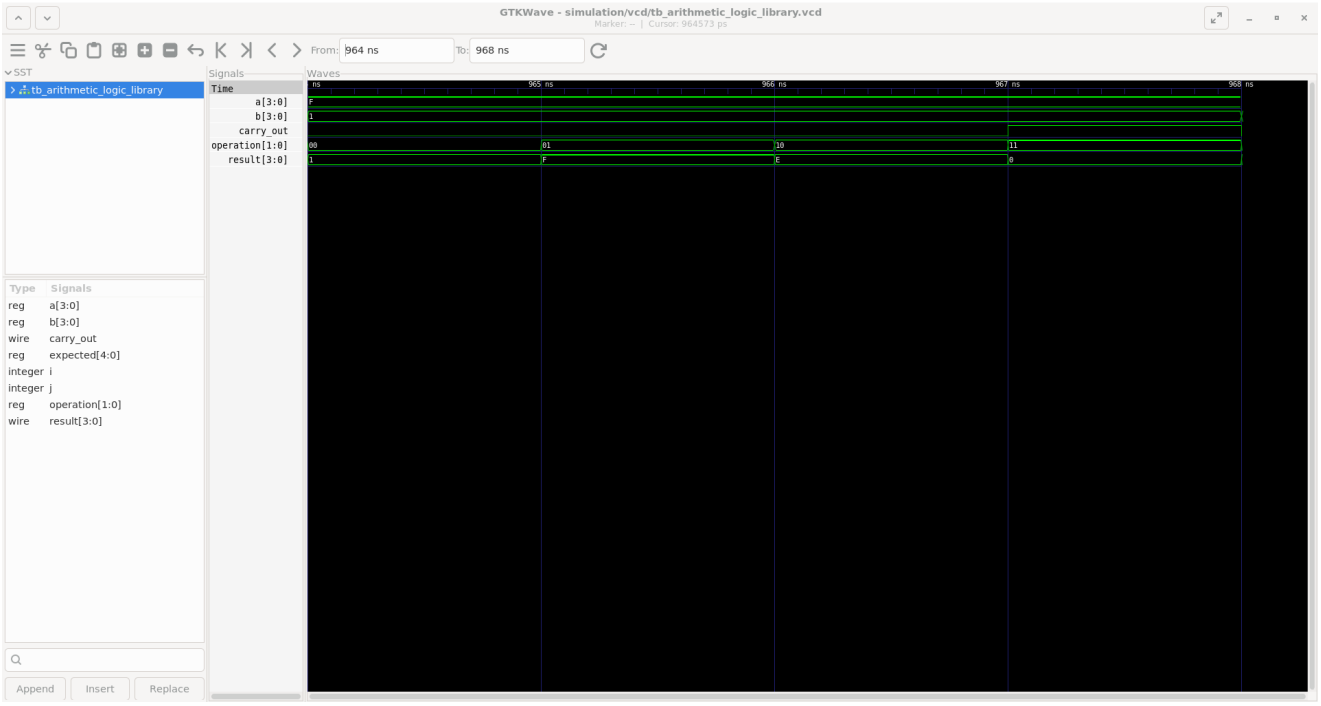
The testbenches use self-checking comparisons and terminate with a fatal error on a mismatch. The suite was run in WSL with Icarus Verilog on 2026-11-01.

Testbench	Coverage	Result
tb_basic_gates	All 4 two-input combinations	PASS
tb_muxes	8 2:1 input/select cases; 4 4:1 selections	PASS
tb_adders	All half-adder and full-adder inputs	PASS
tb_ripple_carry_adder_4bit	All 512 operand/carry-in combinations	PASS
tb_parameterized_adder	1,024 deterministic 8-bit cases	PASS
tb_arithmetic_logic_library	All 4 operations across 16 x 16 operands	PASS

The corresponding simulator logs are in `simulation/results/`; VCD traces are in `simulation/vcd/`. The run script is `scripts/run_all.sh` and returns a failure status if compilation or an assertion fails.

7. GTKWave waveform evidence

The mini-project trace was inspected from 964 ns to 968 ns with a=F and b=1. The operation select steps through AND, OR, XOR, and addition. For operation 11 (addition), the observed result is 0 and carry_out is 1, demonstrating 4-bit unsigned overflow.



8. Environment evidence and conclusion

The student reported Ubuntu 26.04.1 LTS (release 26.04, codename resolute), Icarus Verilog 12.0, VVP 12.0, GTKWave 3.3.126, Git 2.53.0, and VS Code 1.138.0 (x64) on 2026-11-01. The Linux practice report records observed `pwd`, `ls`, and Git commands used to inspect and update the repository. The listed command outcomes are available in linux_report/command_practice.md.

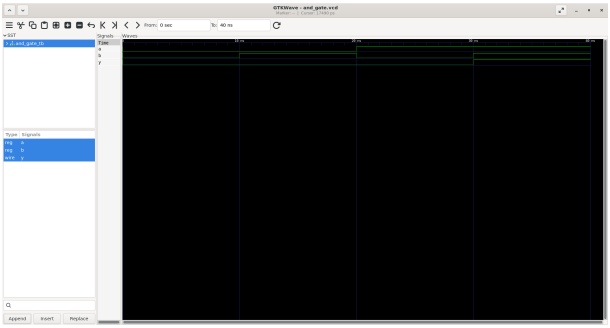
The RTL source and self-checking verification suite are prepared and all six testbenches passed. Seven genuine GTKWave screenshots are included across the mini-project, gates, adders, and 2:1 mux. The internship form requests a separate LinkedIn post URL, which must be supplied outside this repository.

Repository checklist

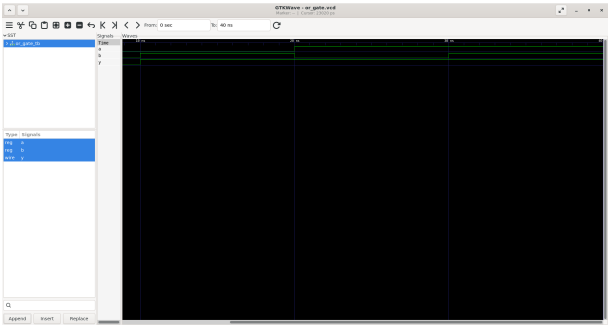
Deliverable	Status
RTL and testbench source	Prepared; all six testbenches passed
Simulation logs and VCD traces	Included from the recorded WSL run
Logic/K-map/design notes and workflow diagrams	Included in docs/
GTKWave screenshot evidence	Seven screenshots: mini-project, gates, adders, and 2:1 mux
Linux environment practice evidence	Ubuntu/tool versions and listed command outcomes recorded

9. Additional GTKWave screenshots

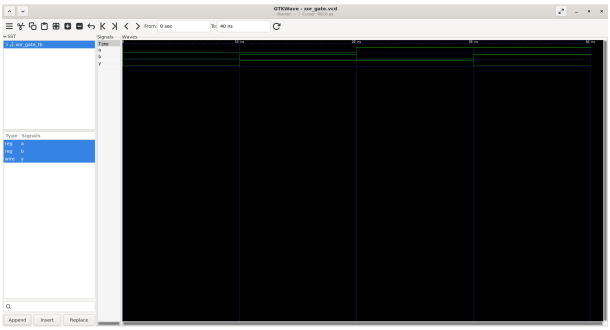
These genuine screenshots were supplied from earlier GTKWave runs and complement the mini-project overflow capture on the previous page.



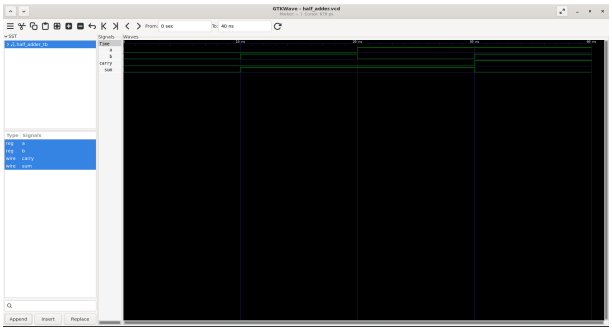
AND gate: a, b, and y across the four input combinations.



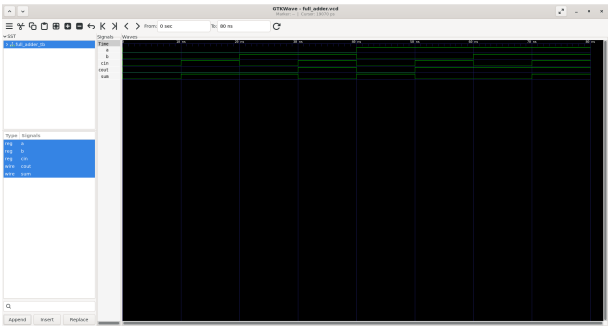
OR gate: output remains high when either input is high.



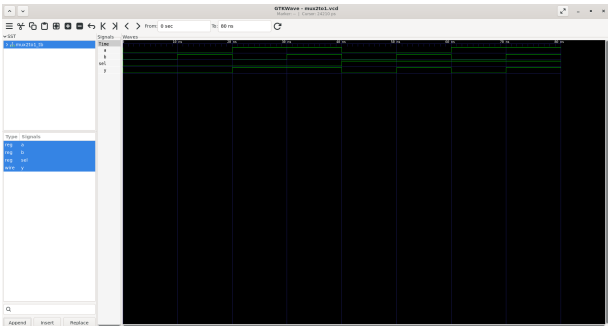
XOR gate: output is high when the inputs differ.



Half adder: sum and carry outputs.



Full adder: sum and carry-out across all input combinations.



2:1 mux: output follows the selected data input.